UTA SUMO
ROBOTICS CUP 2026

Guía completa de instalación, migración y uso
Windows y Linux | FastAPI | React | SQL Server

Desde descargar el repositorio hasta crear la base, iniciar la aplicación y administrar el evento

Manual operativo para desarrollo local. Los comandos, rutas y dependencias se revisaron contra el proyecto actual el 27 de octubre de 2026.

Incluye instalación de herramientas, claves, SQL Server, migraciones, cuentas administrativas, inicio de backend/frontend, roles y flujos principales.

1. Arquitectura y requisitos

El navegador carga React y Vite. La API FastAPI valida identidad, permisos y reglas de negocio. SQL Server guarda usuarios, eventos, categorías, equipos, robots, inscripciones, pagos, historial y reportes. HTTP transporta las operaciones normales; WebSockets mantiene actualizados el marcador y el panel administrativo.

Parte	Tecnología	Uso
Frontend	React 19, Vite 8, Node.js y npm	Interfaz pública, área participante, panel administrativo, marcador.
Backend	Python 3.14, FastAPI, Uvicorn, SQLAlchemy, pyodbc	API REST, autenticación JWT, reglas, reportes, conexiones en tiempo real.
Base de datos	Microsoft SQL Server y ODBC Driver 18	Datos relacionales, índices, restricciones y migraciones.

Requisitos

- Git y acceso al repositorio.

- Python 3.14 para el entorno backend.

- Node.js 20.19+ o 22.12+; se recomienda Node 22 LTS para Vite 8.

- Una instancia SQL Server accesible y Microsoft ODBC Driver 18.

- En Windows puede usarse autenticación integrada. En Linux configure autenticación SQL y permisos de creación de base y tablas.

Windows: instalar herramientas con winget

PowerShell

```
winget install --id Git.Git -e
winget install --id Python.Python.3.14 -e
winget install --id OpenJS.NodeJS.LTS -e
```

Linux Ubuntu: herramientas del sistema y ODBC Driver 18

Instalación Ubuntu (usa las instrucciones oficiales para tu versión/distribución)

```
sudo apt update
sudo apt install -y git curl ca-certificates build-essential python3 python3-venv python3-pip unixodbc-dev
. /etc/os-release
curl -sSL -O "https://packages.microsoft.com/config/ubuntu/${VERSION_ID}/packages-microsoft-prod.deb"
sudo dpkg -i packages-microsoft-prod.deb && rm packages-microsoft-prod.deb
sudo apt update
sudo ACCEPT_EULA=Y apt install -y msodbcsql18
```

Node 22 en Linux mediante NVM

Bash

```
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.40.3/install.sh | bash
source ~/.bashrc
nvm install 22
nvm use 22
```

ODBC para Windows | ODBC para Linux | SQL Server | Node.js

2. Descargar y preparar el repositorio

Ejecuta desde el directorio donde guardarás el proyecto.

Windows

PowerShell

```
cd $HOME\Documents
git clone https://github.com/Deivid05032004/ProyectoNexoraSumoBox.git ProyectoSumoBox
cd ProyectoSumoBox
```

Linux

Bash

```
mkdir -p ~/proyectos && cd ~/proyectos
git clone https://github.com/Deivid05032004/ProyectoNexoraSumoBox.git ProyectoSumoBox
cd ProyectoSumoBox
```

Para usar una rama publicada concreta, lista las ramas y cámbiala si existe:

Cambiar de rama

```
git branch -a
git switch feature/backend-core
```

Estructura principal

Carpeta	Contenido
backend/uta_sumo_api	API, requirements.txt, scripts de operación y .env privado.
frontend/uta-sumo-web	Aplicación web, package-lock.json y .env del frontend.
database/scripts	Esquema SQL y migraciones numeradas; actualmente llegan hasta la 35.
database/seeds	Roles y estados iniciales.
backend/uta_sumo_api/postman	Colecciones HTTP para pruebas manuales y E2E.

Opcional: instalar el motor SQL Server en Ubuntu

Solo si no dispones de un SQL Server. Los comandos siguientes admiten Ubuntu 22.04 con SQL Server 2022 y Ubuntu 24.04 con SQL Server 2025. La configuración interactiva permite elegir edición (Developer y Express son gratuitas) y contraseña del administrador sa.

Servidor SQL en Ubuntu

```
. /etc/os-release
curl -fsSL https://packages.microsoft.com/keys/microsoft.asc |
  sudo tee /etc/apt/trusted.gpg.d/microsoft.asc
repo=https://packages.microsoft.com/config/ubuntu
if [ "$VERSION_ID" = "22.04" ]; then
  curl -fsSL "$repo/22.04/mssql-server-2022.list" \
    | sudo tee /etc/apt/sources.list.d/mssql-server.list
elif [ "$VERSION_ID" = "24.04" ]; then
  curl -fsSL "$repo/24.04/mssql-server-2025.list" \
    | sudo tee /etc/apt/sources.list.d/mssql-server-2025.list
else
  echo "Usa la guía oficial de Microsoft para esta versión de Ubuntu."
fi
sudo apt update
sudo apt install -y mssql-server
sudo /opt/mssql/bin/mssql-conf setup
sudo systemctl enable --now mssql-server
```

Microsoft: instalar SQL Server en Ubuntu

3. Configurar el entorno local

Desde la raíz, copia las plantillas. Completa los secretos localmente; los archivos .env están excluidos de Git.

Crear archivos locales

```
# Windows PowerShell
Copy-Item .env.example backend\uta_sumo_api\.env
Copy-Item frontend\uta-sumo-web\.env.example frontend\uta-sumo-web\.env

# Linux Bash
cp .env.example backend/uta_sumo_api/.env
cp frontend/uta-sumo-web/.env.example frontend/uta-sumo-web/.env
chmod 600 backend/uta_sumo_api/.env frontend/uta-sumo-web/.env
```

Generar claves nuevas

Generar SECRET_KEY y PII_ENCRYPTION_KEY

```
# Windows PowerShell - valor para SECRET_KEY
py -3.14 -c "import secrets; print(secrets.token_urlsafe(48))"
# Windows PowerShell - valor para PII_ENCRYPTION_KEY
py -3.14 -c "import secrets,base64; print(base64.urlsafe_b64encode(secrets.token_bytes(32)).decode().rstrip('='))"

# Linux Bash - valor para SECRET_KEY
python3 -c 'import secrets; print(secrets.token_urlsafe(48))'
# Linux Bash - valor para PII_ENCRYPTION_KEY
python3 -c 'import secrets,base64; print(base64.urlsafe_b64encode(secrets.token_bytes(32)).decode().rstrip("="))'
```

Copia la salida a backend/uta_sumo_api/.env. No reemplaces estas claves después de guardar información cifrada: PII_ENCRYPTION_KEY permite leer datos y archivos protegidos; SECRET_KEY valida JWT. Consérvalas en un gestor de secretos y en backups seguros.

La plantilla raíz configura SQL Server local .\SQLEXPRESS, base UTA_SUMO_ROBOTICS_CUP y autenticación Windows. En Linux usa DB_AUTH_MODE=sql, DB_SERVER=HOST,PUERTO, DB_USERNAME y DB_PASSWORD. El usuario debe poder crear la base en una instalación nueva.

GOOGLE_CLIENT_ID del backend debe coincidir con VITE_GOOGLE_CLIENT_ID del frontend. El SMTP es opcional y sus claves solo van en el .env del backend.

4. Instalar dependencias del backend

Ejecuta desde backend/uta_sumo_api. Se usa un entorno virtual para no instalar paquetes Python globalmente.

Instalar Python, FastAPI y conectores SQL

```
# Windows PowerShell
cd backend\uta_sumo_api
py -3.14 -m venv .venv
.venv\Scripts\python.exe -m pip install --upgrade pip
.venv\Scripts\python.exe -m pip install -r requirements.txt

# Linux Bash
cd backend/uta_sumo_api
python3 -m venv .venv
. .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
```

El requirements.txt instala FastAPI, Uvicorn, SQLAlchemy, pyodbc, autenticación, cifrado, validación de correo, carga de archivos y dependencias de pruebas/PDF. Comprueba que el conector aparece:

```
# Windows
py -3.14 -c "import pyodbc; print(pyodbc.drivers())"
# Linux (con .venv activado)
python -c 'import pyodbc; print(pyodbc.drivers())'
# Debe aparecer: ODBC Driver 18 for SQL Server
```

5. Migrar toda la base de datos

El comando se ejecuta desde backend/uta_sumo_api. Crea la base si no existe y aplica las migraciones en orden; en una base existente omite las ya registradas. No borra inscripciones, usuarios ni pagos. La migración 35 normaliza el evento oficial como Programado y activo; la 36 incorpora el cambio obligatorio de contraseña inicial para la cuenta ADMIN local.

Migración completa y aditiva

```
# Windows
cd backend\uta_sumo_api
.venv\Scripts\python.exe scripts\migrate.py

# Linux
cd backend/uta_sumo_api
.venv/bin/python scripts/migrate.py
```

> No uses --reset-local --confirm-drop como migración normal: esa opción elimina la base local. El comando de arriba es el correcto para máquina nueva y para actualizar preservando datos.

Cuentas demo automáticas

La plantilla raíz deja AUTO_BOOTSTRAP_DEMO_ACCOUNTS=true. En una instancia local, el comando migrate crea los perfiles de prueba faltantes con contraseñas aleatorias y las imprime una sola vez en la terminal. Si se migran otra vez y ya existen, sus claves no se vuelven a imprimir. La protección impide crear perfiles demo en una base remota/productiva. También crea la cuenta ADMIN local descrita en la siguiente sección si no existe.

6. Cuentas de administrador y acceso

Cuenta ADMIN local fija: adminTics@uta.edu.ec. Contraseña inicial: Admin@utaTics. El bootstrap la crea solo si falta y no restablece la clave en migraciones posteriores. SQL conserva solo el hash Argon2id.

Solo para desarrollo local: la clave inicial tiene 13 caracteres, menos que el mínimo general de 15. No uses esta cuenta en servidores compartidos o productivos; el bootstrap está limitado a la base local UTA_SUMO_ROBOTICS_CUP.

En el primer acceso, la API informa must_change_password=true y el panel abre Seguridad de la cuenta, bloqueando el resto de las funciones administrativas. Cambia la clave inicial por una frase de 15 caracteres o más que no sea común, predecible ni igual a la anterior. El cambio se audita, revoca las sesiones y requiere iniciar sesión de nuevo.

Todo ADMIN puede cambiar luego su clave desde Seguridad de la cuenta mediante POST /api/v1/auth/change-password. Se requiere la contraseña actual; no es recuperación por correo.

El bootstrap también prepara admin.demo@example.com, administrator.demo@example.com, event_manager.demo@example.com y participant.demo@example.com. Sus claves aleatorias se imprimen solo al crearlas y no se pueden recuperar de SQL.

Si pierdes una clave demo aleatoria, puedes rotar las cuentas locales; esto no restablece la cuenta fija adminTics@uta.edu.ec.

Comando para volver a obtener credenciales demo

```
# Windows (desde backend/uta_sumo_api)
.venv\Scripts\python.exe scripts\bootstrap_local_demo_accounts.py --rotate-existing

# Linux
.venv/bin/python scripts/bootstrap_local_demo_accounts.py --rotate-existing
```

No ejecutes la rotación en una base real. No compartas el resultado por chat ni lo guardes en el repositorio. Para rotar y probar el flujo E2E con Newman (API encendida y npm disponible):

Rotar cuentas demo y ejecutar flujo automatizado

```
# Windows
.venv\Scripts\python.exe scripts\bootstrap_local_demo_accounts.py --rotate-existing --run-postman
# Linux
.venv/bin/python scripts/bootstrap_local_demo_accounts.py --rotate-existing --run-postman
```

Crear cuenta real de operador

Para uso real, crea un operador en vez de usar las cuentas demo. El script pide correo, nombre, rol ADMIN o EVENT_MANAGER y contraseña (mínimo 15 caracteres). La contraseña la eliges durante el proceso y no se imprime.

Aprovisionar operador desde backend/uta_sumo_api

```
# Windows
.venv\Scripts\python.exe scripts\provision_operator_user.py
# Linux
.venv/bin/python scripts/provision_operator_user.py
```

7. Iniciar el backend

Mantén esta terminal abierta. Abre Swagger para probar las rutas.

FastAPI / Uvicorn

```
# Windows PowerShell, desde backend/uta_sumo_api
.venv\Scripts\python.exe -m uvicorn main:app --host 127.0.0.1 --port 8000 --reload

# Linux Bash, desde backend/uta_sumo_api
.venv/bin/python -m uvicorn main:app --host 127.0.0.1 --port 8000 --reload
```

API: http://127.0.0.1:8000 | Swagger: http://127.0.0.1:8000/docs | OpenAPI: /openapi.json

Comprueba DB_SERVER, permisos y ODBC si no conecta SQL. Si sale clave de cifrado faltante, revisa .env. En producción usa HTTPS, secretos del servidor, proxy y sin --reload.

8. Instalar e iniciar el frontend

Abre una segunda terminal independiente y ejecuta desde frontend/uta-sumo-web. npm ci usa el package-lock.json para instalar versiones reproducibles.

React / Vite

```
# Windows PowerShell
cd frontend\uta-sumo-web
Copy-Item .env.example .env
npm ci
npm run dev -- --host 127.0.0.1

# Linux Bash
cd frontend/uta-sumo-web
cp .env.example .env
npm ci
npm run dev -- --host 127.0.0.1
```

Abre http://127.0.0.1:5173. Si Windows bloquea npm por ExecutionPolicy, usa npm.cmd ci y npm.cmd run dev -- --host 127.0.0.1; no es necesario cambiar la política global. Cambia el .env solo con Vite detenido y reinícialo después.

Comprobar compilación

Build de producción

```
# Windows PowerShell
npm.cmd run build
# Linux
npm run build
```

9. Programar eventos y entender los estados

Entra al panel con ADMIN. El selector del panel principal muestra eventos activos y define el evento cuyo panel de inscripciones, pagos y comprobantes estás viendo. En Gestión de eventos puedes administrar borradores e inactivos también.

Control	Para qué sirve
Evento	Nombre, año, sede y fechas de inscripción/competencia.
Categorías y cupos	Asociar categorías, límites de participantes/coaches, cupos y tarifa. Los mínimos/máximos deben ser coherentes; precios no pueden ser negativos.
Formulario	Editar preguntas. Publicar una versión no altera respuestas anteriores.
Reglamento	Cargar un PDF, administrar versiones y publicar la versión vigente.
Historial	Consultar cambios administrativos y estados.

Ciclo del evento

Estado	Botón	Significado
Borrador	Programar evento	Preparación; todavía no está programado.
Programado	Iniciar competencia	Activo y listo. Las fechas/categorías/tarifas determinan si se admiten inscripciones.
En curso	Pausar o Finalizar	Competencia en ejecución.
Pausado	Reanudar o Finalizar	Competencia detenida temporalmente.
Finalizado / Cancelado	Sin transición	Estados terminales.

Fechas: inicio y cierre de inscripciones definen la ventana de registro. Cierre vacío significa que no se fijó una fecha límite. Inicio y fin del evento son el calendario de la competencia; el cierre de inscripción no debe quedar después del inicio del evento. El evento oficial UTA SUMO ROBOTICS CUP 2026 quedó activo y Programado; completa las fechas del torneo cuando la organización las confirme.

10. Flujos para participantes, administradores y jueces

Inscripción

- Participante se autentica con Google y registra equipo, robot y roster. No se pide ni se guarda la contraseña de Google.

- El backend verifica propiedad, duplicados, categoría, cupos, participantes/coaches y tarifa vigente.

- La inscripción, el roster, pago pendiente e historial se crean en una transacción SQL. La API pagina listados desde la base.

Pago

- Participante carga un recibo PDF, PNG o JPEG bajo el límite configurado; el backend lo valida, cifra y guarda de forma privada.

- ADMIN revisa el comprobante, aprueba o rechaza el pago; se conserva auditoría. El panel se actualiza por WebSocket y reconciliación.

Marcador y reportes

- ADMIN/EVENT_MANAGER administran rondas, enfrentamientos, estado y puntajes append-only; los ajustes negativos requieren motivo y no dejan total menor a cero.

- La pantalla pública recibe puntaje, cronómetro y estado en vivo. El timer puede contar hacia delante o hacia atrás.

- Reportes: resumen, inscripciones por estado/categoría, tendencia y resumen de pagos. Los detalles son paginados; los reportes financieros requieren ADMIN.

11. Pruebas y solución de errores

Pruebas unitarias desde backend/uta_sumo_api

Suite unitaria y SQL

```
# Windows
.venv\Scripts\python.exe -m pytest -q
# Linux
.venv/bin/python -m pytest -q

# Las pruebas SQL requieren una base de pruebas controlada
# PowerShell
$env:RUN_SQLSERVER_TESTS='1'; .venv\Scripts\python.exe -m pytest -q
# Bash
RUN_SQLSERVER_TESTS=1 .venv/bin/python -m pytest -q
```

Error	Revisión
API no inicia	SECRET_KEY y PII_ENCRYPTION_KEY; activa ODBC 18; revisa conexión SQL.
No conecta a SQL	Instancia y puerto, permisos, firewall y modo de autenticación; Linux usa DB_AUTH_MODE=sql.
Frontend Failed to fetch	Mantén backend en puerto 8000; verifica VITE_API_BASE_URL y CORS_ORIGINS.
405 Method Not Allowed	Reinicia el backend desde la carpeta correcta y recarga con Ctrl+F5.
422 Unprocessable Content	Revisa requeridos, formato, fechas, límites de cupo y tarifa no negativa.
No aparecen claves demo	Solo se muestran al crearlas/rotarlas; no se recuperan de hashes.

Postman

Importa backend/uta_sumo_api/postman/UTA_Sumo_API.postman_collection.json y el environment local. Introduce las credenciales que imprimió la creación/rotación demo. No subas tokens, claves o environments privados a Git.

12. Referencia rápida de terminales

Acción	Windows	Linux
Migrar base	Desde backend/uta_sumo_api: .venv\Scripts\python.exe scripts\migrate.py	Desde backend/uta_sumo_api: .venv/bin/python scripts/migrate.py
Iniciar API	Desde backend/uta_sumo_api: .venv\Scripts\python.exe -m uvicorn main:app --host 127.0.0.1 --port 8000 --reload	Desde backend/uta_sumo_api: .venv/bin/python -m uvicorn main:app --host 127.0.0.1 --port 8000 --reload
Instalar web	Desde frontend/uta-sumo-web: npm ci	Desde frontend/uta-sumo-web: npm ci
Iniciar web	Desde frontend/uta-sumo-web: npm.cmd run dev -- --host 127.0.0.1	Desde frontend/uta-sumo-web: npm run dev -- --host 127.0.0.1
Rotar demo	Desde backend: .venv\Scripts\python.exe scripts\bootstrap_local_demo_accounts.py --rotate-existing	Desde backend: .venv/bin/python scripts/bootstrap_local_demo_accounts.py --rotate-existing
Crear operador	Desde backend: .venv\Scripts\python.exe scripts\provision_operator_user.py	Desde backend: .venv/bin/python scripts/provision_operator_user.py

Siempre usa dos terminales

- Terminal 1: backend/uta_sumo_api; migra una vez y luego deja Uvicorn ejecutándose.

- Terminal 2: frontend/uta-sumo-web; instala con npm ci una vez y deja Vite ejecutándose.

- Abre la aplicación en http://127.0.0.1:5173 y la documentación API en http://127.0.0.1:8000/docs.

Referencias oficiales

Microsoft ODBC Driver
Instalar ODBC en Linux
Node.js
Python
Repositorio

Seguridad: deshabilita las cuentas demo en producción, conserva secretos y backups cifrados, crea operadores reales por canal seguro y no expongas SQL Server directamente a Internet.